



МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»



**НГТУ
НЭТИ**

**Факультет прикладной
математики
и информатики**

Кафедра прикладной математики
Практическое задание № 2
по дисциплине «Цифровые модели и оценивание параметров»

НЕЛИНЕЙНЫЕ ОБРАТНЫЕ ЗАДАЧИ

ПМ-34 КУДЮКОВ МАКСИМ
Вариант 4 ПМ-34 РУБЦОВ АРТЁМ

Преподаватель ВАГИН ДЕНИС ВЛАДИМИРОВИЧ

Новосибирск, 2026

Содержание

1 Задание.....	3
2 Постановка задачи	3
3 Входные данные	3
4 Теоретическая часть	3
5 Тестирование.....	4
5.1 Тестирование 0	4
5.2 Тестирование 1	4
5.3 Тестирование 2	4
5.4 Тестирование 3	5
6 Вывод.....	5
7 Программа	5

1 Задание

Реализовать решение обратной задачи с выбранным вариантом.

2 Постановка задачи

Однородное полупространство. Приёмники 1–3. Источник 2.

Определить значение σ полупространства. Добавить шум, равный 10 % от значения измерения. Не использовать веса w_i .

3 Входные данные

Положение источников	Положение приёмников
A2(0,0,0); B2(100,0,0);	M1(200,0,0), N1(300,0,0); M2(500,0,0), N2(600,0,0); M3(1000,0,0), N3(1100,0,0).

4 Теоретическая часть

$$\Phi(\sigma) = \sum_i^{N^r} (w_i(V_i(\sigma) - \bar{V}_i))^2$$

$$V_{AB}^{MN} = \frac{I}{2\pi\sigma} \left(\left(\frac{1}{r_B^M} - \frac{1}{r_A^M} \right) - \left(\frac{1}{r_B^N} - \frac{1}{r_A^N} \right) \right).$$

Где r_B^M – расстояние между источником B и приёмником M
(у нас в задаче 3 приёмника и 1 источник поэтому M и B будут иметь индексы)

$$r_{B_2}^{M_i}, r_{B_2}^{N_i}, r_{A_2}^{M_i}, r_{A_2}^{N_i} \quad i = 1, 2, 3$$

Задача решается через метод Гаусса-Ньютона, поэтому нужно собрать систему и решить СЛАУ

$$A\Delta\sigma = b, \text{ система, параметризуем } \sigma, L = \ln \sigma, \frac{\partial V_i}{\partial L} = -V_i \Rightarrow A\Delta L = b$$

$$A_{qs} = \sum_i^{N^r} (w_i)^2 \left(\frac{\partial(\delta V_i(\sigma^k))}{\partial \sigma} \right)^2 \Rightarrow \sum_i^{N^r} (w_i)^2 (V_i)^2$$

$$b_q = \sum_i^{N^r} (w_i)^2 \delta V_i(\sigma^k) \frac{\partial(\delta V_i(\sigma^k))}{\partial \sigma} \Rightarrow - \sum_i^{N^r} (w_i)^2 (V_i - \bar{V}_i) V_i$$

5 Тестирование

5.1 Тестирование 0

$\sigma_0 = 0.1$; $\sigma_{true} = 1.0$; $I = 1$; $\varepsilon = 10^{-16}$; $\delta = 10^{-6}$; Шум = 0%

Итерация	σ	Total_Residual	Phi1	Phi2	Phi3
0	0.1	2.29E-05	2.28E-05	5.70E-08	8.37E-10
1	0	0.00	0.00	0.00	9.72E-11
2	0.522806	2.35E-07	2.34E-07	5.86E-10	8.61E-12
3	0.842526	9.86E-09	9.83E-09	2.46E-11	3.61E-13
4	0.986219	5.51E-11	5.50E-11	1.37E-13	2.02E-15
5	0.999904	2.59E-15	2.59E-15	6.46E-18	9.50E-20
6	1	5.95E-24	5.94E-24	1.48E-26	2.18E-28

5.2 Тестирование 1

$\sigma_0 = 0.1$; $\sigma_{true} = 1.0$; $I = 1$; $\varepsilon = 10^{-10}$; $\delta = 10^{-6}$; Шум = $\pm 10\%$

Итерация	σ	Total_Residual	Phi1	Phi2	Phi3
0	0.1	2.54E-07	2.54E-07	2.67E-10	2.62E-12
1	0.09	0.00	0.00	0.00	0.00
2	0.091364	1.72E-09	4.11E-12	1.72E-09	2.02E-12
3	0.091365	1.72E-09	4.32E-12	1.71E-09	2.02E-12

5.3 Тестирование 2

$\sigma_0 = 1.75$; $\sigma_{true} = 0.25$; $I = 1$; $\varepsilon = 10^{-10}$; $\delta = 10^{-6}$; Шум = $\pm 10\%$

Итерация	σ	Total Residual	Phi1	Phi2	Phi3
0	1.75	2.63E-06	2.62E-06	7.42E-09	9.74E-11
1	0.01	0.00	0.00	0.00	0.00
2	0.022092	0.000489	0.000488	1.21E-06	1.79E-08
3	0.055433	5.86E-05	5.85E-05	1.42E-07	2.14E-09
4	0.123259	5.69E-06	5.67E-06	1.30E-08	2.07E-10
5	0.214351	3.06E-07	3.06E-07	5.05E-10	1.09E-11
6	0.267959	3.32E-09	3.31E-09	5.21E-12	8.65E-14
7	0.27583	2.72E-11	1.17E-12	2.61E-11	2.33E-15

Логарифмическая параметризация не дала уйти в минус.

5.4 Тестирование 3

$\sigma_0 = 0.01$; $\sigma_{true} = 0.1$; $I = 1$; $\varepsilon = 10^{-10}$; $\delta = 10^{-6}$; Шум = $\pm 10\%$

Итерация	σ	Total Residual	Phi1	Phi2	Phi3
0	0.01	0.002329	0.002323	5.82E-06	8.45E-08
1	0.02	0.00	2.73E-04	6.89E-07	9.76E-09
2	0.05375	2.53E-05	2.52E-05	6.47E-08	8.41E-10
3	0.08938	1.18E-06	1.18E-06	3.33E-09	2.65E-11
4	0.107307	8.76E-09	8.69E-09	6.56E-11	7.42E-13
5	0.109353	1.46E-11	5.09E-13	1.21E-11	2.02E-12

6 Вывод

Алгоритм решает задачу во всех трёх сценариях за 3–7 итераций. Логарифмическая параметризация обеспечивает устойчивость при грубом начальном приближении. Отклонение итоговой σ от истины на 5–10 % объясняется шумом $\pm 10\%$ и точностью поиска решения.

7 Программа

```
//main.cpp
```

```
#include "ReverseProblem.h"
#include <ctime>
```

```
int main()
{
    srand(static_cast<unsigned>(time(nullptr)));

    ReverseProblem problem(3, 1.0);
    problem.lnSigma = std::log(0.1);
    problem.sigma = 0.1;
    problem.eps = 1e-15;
    try
    {
        problem.Solve();
    }
    catch (const std::runtime_error& e)
    {
        std::cout << "Caught a runtime error: " << e.what() << std::endl;
    }

    std::cout << "Final sigma = " << problem.sigma << std::endl;

    return 0;
}
```

```
//ReverseProblem.h
#pragma once
```

```
#include <vector>
#include <iostream>
#include <cmath>
#include <numbers>
```

```

class ReverseProblem
{
public:
    ReverseProblem( int _n, double _trueSigma)
    {
        n = _n;
        trueSigma = _trueSigma;
        lnSigma = std::log(0.01);
        sigma = 0.01;
        A.resize( 1, std::vector<double>( 1, 0 ) );
        b.resize( 1 );
        dVdSigma.resize( n, 0.0 );
        trueV.resize( n );
        V.resize( n );
        w2.resize( n );
        residuals.resize( n );
        receiverMPos = { xyz( 200,0,0 ), xyz( 500,0,0 ), xyz( 1000,0,0 ) };
        receiverNPos = { xyz( 300,0,0 ), xyz( 600,0,0 ), xyz( 1100,0,0 ) };
        sourceAPos = { xyz( 0,-500,0 ), xyz( 0,0,0 ), xyz( 0,500,0 ) };
        sourceBPos = { xyz( 100,-500,0 ), xyz( 100,0,0 ), xyz( 100,500,0 ) };
    }

    struct xyz {

        xyz( ) : x( 0 ), y( 0 ), z( 0 ) {}

        xyz( double _x, double _y, double _z ) : x( _x ), y( _y ), z( _z ) {}

        double x = 0;
        double y = 0;
        double z = 0;
    };

    int n = 0;
    std::vector<std::vector<double>> A;
    std::vector<double> b;
    std::vector<double> trueV;
    std::vector<double> V;
    std::vector<xyz> receiverMPos;
    std::vector<xyz> receiverNPos;
    std::vector<xyz> sourceAPos;
    std::vector<xyz> sourceBPos;
    double lnSigma;
    double sigma;
    double trueSigma;
    std::vector<double> dVdSigma;
    std::vector<double> w2;
    std::vector<double> residuals;
    double alpha = 0.0;
    double eps = 1e-10;
    int maxIter = 10;

    double Distance( xyz p1, xyz p2 );
    void ComputeTrueV( );
    void ComputeCurrentV( );
    void ComputeJacobian( );
    void WeightsInit( );
    void BuildMatrix( );
    void BuildRightPart( );
    void Regularisation( );
    double CalcResidual( );
    void CalcIndividualResiduals( );

```

```

    void Solve( );

};

//ReverseProblem.cpp

#include "ReverseProblem.h"
#include "Gauss.h"

double ReverseProblem::Distance(xyz p1, xyz p2)
{
    return sqrt((p2.x - p1.x) * (p2.x - p1.x) +
                (p2.y - p1.y) * (p2.y - p1.y) +
                (p2.z - p1.z) * (p2.z - p1.z));
}

void ReverseProblem::ComputeTrueV()
{
    double piSigma = 0.5 * 1 / (std::numbers::pi * trueSigma);
    for (int i = 0; i < n; i++)
    {
        double geom = 1.0 / Distance(sourceBPos[1], receiverMPos[i])
            - 1.0 / Distance(sourceAPos[1], receiverMPos[i])
            - (1.0 / Distance(sourceBPos[1], receiverNPos[i])
            - 1.0 / Distance(sourceAPos[1], receiverNPos[i]));
        trueV[i] = piSigma * geom;
        double noise = 0.1 * (2.0 * rand() / RAND_MAX - 1.0); // случайное в [-0.1,
0.1]
        //trueV[i] *= (1.0 + noise);
    }
}

void ReverseProblem::ComputeCurrentV()
{
    sigma = std::exp(lnSigma);
    double piSigma = 0.5 * 1 / (std::numbers::pi * sigma);
    for (int i = 0; i < n; i++)
    {
        double geom = 1.0 / Distance(sourceBPos[1], receiverMPos[i])
            - 1.0 / Distance(sourceAPos[1], receiverMPos[i])
            - (1.0 / Distance(sourceBPos[1], receiverNPos[i])
            - 1.0 / Distance(sourceAPos[1], receiverNPos[i]));

        V[i] = piSigma * geom;
    }
}

void ReverseProblem::ComputeJacobian()
{
    sigma = std::exp(lnSigma);
    ComputeCurrentV();
    for (int i = 0; i < n; i++)
        dVdSigma[i] = -V[i];
    double piSigma = 0.5 * 1 / (std::numbers::pi * sigma);
    for (int i = 0; i < n; i++)
    {
        double geom = 1.0 / Distance(sourceBPos[1], receiverMPos[i])
            - 1.0 / Distance(sourceAPos[1], receiverMPos[i])
            - (1.0 / Distance(sourceBPos[1], receiverNPos[i])
            - 1.0 / Distance(sourceAPos[1], receiverNPos[i])

```

```

        - 1.0 / Distance(sourceAPos[1], receiverNPos[i]));

        // dV_i/dσ = - geom / (2π σ²)
    }
}

void ReverseProblem::WeightsInit()
{
    for (int i = 0; i < n; i++)
        w2[i] = 1;
}

void ReverseProblem::BuildMatrix()
{
    A[0][0] = 0.0;
    for (int i = 0; i < n; i++)
        A[0][0] += w2[i] * dVdSigma[i] * dVdSigma[i];
}

void ReverseProblem::BuildRightPart()
{
    b[0] = 0.0;
    for (int i = 0; i < n; i++)
        b[0] += w2[i] * (trueV[i] - V[i]) * dVdSigma[i];
}

double ReverseProblem::CalcResidual()
{
    double res = 0;
    for (int i = 0; i < n; i++)
    {
        double diff = V[i] - trueV[i];
        res += w2[i] * diff * diff;
    }
    return res;
}

void ReverseProblem::CalcIndividualResiduals()
{
    for (int i = 0; i < n; i++)
    {
        double diff = V[i] - trueV[i];
        residuals[i] = w2[i] * diff * diff;
    }
}

void ReverseProblem::Solve()
{
    double residual = 1;
    Gauss gauss;

    ComputeTrueV();
    WeightsInit();
    ComputeJacobian();

    std::cout << "Iter\tsigma\tTotal_Residual\tPhi1\tPhi2\tPhi3" << std::endl;

    // Начальное приближение sigma задано в конструкторе (например, 0.01)
    ComputeCurrentV();
    residual = CalcResidual();
    CalcIndividualResiduals();
}

```

```

std::cout << 0 << "\t" << sigma << "\t" << residual << "\t";
for (int i = 0; i < n; i++)
{
    std::cout << residuals[i];
    if (i < n - 1) std::cout << "\t";
}
std::cout << std::endl;

for (int iter = 0; iter < maxIter && residual > eps; iter++)
{
    A[0][0] = 0.0;
    b[0] = 0.0;

    ComputeCurrentV();
    ComputeJacobian();
    BuildMatrix();
    BuildRightPart();
    //gauss.printMatrix( A, b );
    //Regularisation();

    std::vector<double> delta = gauss.solve(A, b);
    if (std::abs(delta[0]) < 1e-6) break;
    lnSigma += delta[0];
    sigma = std::exp(lnSigma);

    ComputeCurrentV();
    residual = CalcResidual();
    CalcIndividualResiduals();
    std::cout << iter + 1 << "\t" << sigma << "\t" << residual << "\t";
    for (int i = 0; i < n; i++)
    {
        std::cout << residuals[i];
        if (i < n - 1) std::cout << "\t";
    }
    std::cout << std::endl;
}
}

//Gauss.h

#include <iostream>
#include <vector>
#include <cmath>
#include <stdexcept>

using namespace std;

class Gauss {
public:
    vector<double> solve(vector<vector<double>> A, vector<double> b);
    void printMatrix(const vector<vector<double>>& A, const vector<double>& b);
};

//Gauss.cpp

#include "Gauss.h"

vector<double> Gauss::solve(vector<vector<double>> A, vector<double> b) {
    int n = A.size();

    if (n == 0 || A[0].size() != n || b.size() != n) {
        throw invalid_argument("Неверные размеры матрицы или вектора");
    }
}

```

```

    }

    for (int k = 0; k < n; ++k) {
        int maxRow = k;
        double maxVal = fabs(A[k][k]);

        for (int i = k + 1; i < n; ++i) {
            if (fabs(A[i][k]) > maxVal) {
                maxVal = fabs(A[i][k]);
                maxRow = i;
            }
        }

        if (maxVal < 1e-10) {
            throw runtime_error("Division by zero");
        }

        if (maxRow != k) {
            swap(A[k], A[maxRow]);
            swap(b[k], b[maxRow]);
        }

        double pivot = A[k][k];
        for (int j = k; j < n; ++j) {
            A[k][j] /= pivot;
        }
        b[k] /= pivot;

        for (int i = k + 1; i < n; ++i) {
            double factor = A[i][k];
            for (int j = k; j < n; ++j) {
                A[i][j] -= factor * A[k][j];
            }
            b[i] -= factor * b[k];
        }
    }

    vector<double> x(n, 0.0);
    for (int i = n - 1; i >= 0; --i) {
        x[i] = b[i];
        for (int j = i + 1; j < n; ++j) {
            x[i] -= A[i][j] * x[j];
        }
    }

    return x;
}

void Gauss::printMatrix(const vector<vector<double>>& A, const vector<double>& b) {
    int n = A.size();
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            cout << A[i][j] << "\t";
        }
        cout << "| " << b[i] << endl;
    }
    cout << endl;
}

```